Customer Build Runbook — building a signed delivery on a fresh machine

How to run `packaging/windows-build.ps1` for a **customer** delivery (both `-Customer` and `-CustomerID` set) on a new Windows build host.

Because a customer build auto-sets `WindowsSigningRequired` and `GpgSigningRequired` to `$true`, the script **fails closed** (hard error, not a warning) if any part of the signing chain is missing. Steps below mark the hard blockers and give a substitute for each.

Related: `build-guide.md`, `signing-infrastructure.md`.

Example target command:

```
.\packaging\windows-build.ps1 `
  -Version    "1.8.5" `
  -Customer   "HDFC Bank" `
  -CustomerID "hdfc-prod-001" `
  -Days       311 `
  -WindowsCertThumbprint "<thumbprint>"
```

---

0. Machine requirements

Resource	Minimum	Why
OS	Windows 10/11 x64	PowerShell build + Windows Authenticode signing

RAM	**24 GB** (Docker VM ≥16 GB)	`garble -literals` OOM-kills with less — the common `cannot allocate memory` build failure
Disk	~60 GB free	Docker images (postgres, caldera, headless-shell, orchestrator), Go build cache, two Go toolchains
Privileges	Local admin	installing toolchains, importing certs, Docker
Network	Internet on first build	`docker pull`, caldera emu-lib clone, `go install`, module downloads

---

1. Base tools

1a. Docker Desktop (WSL2 backend) — give the VM RAM

Install Docker Desktop with the WSL2 engine. Create `C:\Users\<you>\.wslconfig`:

```
[wsl2]
memory=18GB
swap=8GB
processors=6
```

Apply with `wsl --shutdown`, then restart Docker Desktop. **Verify:** `docker info` → `Total Memory` ≈ 18 GiB; `docker run --rm hello-world`. **Still OOM:** raise `swap` to 16–20 GB; close other apps; confirm you edited the `.wslconfig` of the account that runs Docker. **Fallback (Hyper-V backend):** Docker Desktop → Settings → Resources → Memory → 18 GB → Apply & Restart (no `.wslconfig`).

1b. Go (1.26.x)

https://go.dev/dl/ — **verify** `go version`.

1c. Git for Windows

https://git-scm.com/download/win — **verify** `git --version`. (Also provides a fallback `gpg.exe`.)

1d. garble (modern)

```
go install mvdan.cc/garble@latest
```

Ensure `"$(go env GOPATH)\bin"` is on `PATH`. **Verify:** `garble version`.

1e. cosign — hard blocker

```
winget install sigstore.cosign
```

Verify: `cosign version`. **Fallbacks:** `go install github.com/sigstore/cosign/v2/cmd/cosign@latest`, or download `cosign-windows-amd64.exe`, rename to `cosign.exe`, put on PATH. **If missing:** build aborts — "cosign is not installed, and cosign signing is mandatory for this customer build."

1f. GPG (Gpg4win) — hard blocker

Install Gpg4win (https://gpg4win.org). **Verify:** `& "C:\Program Files (x86)\GnuPG\bin\gpg.exe" --version`. **Fallback:** the script also finds Git-for-Windows' `gpg` and a plain `gpg` on PATH. On `error running keyboxd`: remove `use-keyboxd` from `%USERPROFILE%\.gnupg\common.conf` and re-import the key.

1g. Legacy agent toolchain — hard blocker

Needs a **real** go1.20.14 GOROOT at `C:\go1.20.14` **and** garble **v0.10.1** at `C:\go1.20.14\gobin\garble.exe`. Provision once:

```
# 1) real go1.20.14 GOROOT
Invoke-WebRequest "https://go.dev/dl/go1.20.14.windows-am
Expand-Archive "$env:TEMP\go12014.zip" -DestinationPath "
if (Test-Path C:\go1.20.14) { Remove-Item -Recurse -Force
Move-Item "$env:TEMP\go12014\go" C:\go1.20.14

# 2) garble v0.10.1 into a DEDICATED gobin, built with th
$env:GOROOT="C:\go1.20.14"; $env:GOTOOLCHAIN="local"; $en
$env:PATH="C:\go1.20.14\bin;$env:PATH"
& C:\go1.20.14\bin\go.exe install mvdan.cc/garble@v0.10.1
$env:GOROOT=""; $env:GOTOOLCHAIN=""; $env:GOBIN=""
```

Verify: `& C:\go1.20.14\bin\go.exe version` → `go1.20.14`; `Test-Path C:\go1.20.14\gobin\garble.exe` → True. **Must-nots:** do not use `$env:GOTOOLCHAIN="go1.20.14"` on top of the main Go (it downloads a *module* toolchain garble cannot build against); `C:\go1.20.14` must be a real directory, not a symlink. Dedicated GOBIN keeps v0.10.1 from clobbering the modern garble at `GOPATH\bin`.

1h. Windows code-signing certificate — hard blocker

Import the Authenticode **code-signing** cert into the Windows store, get the thumbprint:

```
Get-ChildItem Cert:\CurrentUser\My -CodeSigningCert | For
# or Cert:\LocalMachine\My
```

Pass it as `-WindowsCertThumbprint` (or set `$env:BAS_WINDOWS_CERT_THUMBPRINT`). **If missing:** build errors immediately at the agent-signing step; no skip for a customer build.

---

2. Secrets that `git clone` does NOT bring

These are untracked — copy them **securely** into the same relative paths:

Path	Role	
`orchestrator\private_key.pem`	RSA-4096 scenario signing	a new key diverges fr whole line)
`packaging\licensing\keys\private.pem`	signs `<customer-id>.lic`	**No valid s** embedded `orchestr` or the clien
`packaging\signing\cosign.key`	cosign-signs the orchestrator tar	`bash pac` regenerate — don't reg

Plus the **GPG secret key** for `releases@audspect.com` (keyring, not a repo file):

```
# export once on an existing host:
gpg --export-secret-keys --armor releases@audspect.com >
# import on the new host:
gpg --import releases-secret.asc
```

Verify: `gpg --list-secret-keys releases@audspect.com`. Keep these off shared storage; delete `releases-secret.asc` after import. (`cosign.pub`, `pubkey.asc`, `license_key.go` are tracked and arrive with the repo.)

---

3. Get the repository

```
cd C:\Users\<you>\Downloads
git clone <repo-url> Audspect_Cloud
cd Audspect_Cloud
git checkout main   # or the exact release commit/tag to
```

Then drop the three §2 key files into their paths.

---

4. Pre-flight check

```
foreach ($f in @(
  "orchestrator\private_key.pem",
  "packaging\licensing\keys\private.pem",
  "packaging\signing\cosign.key")) {
    "{0,-45} {1}" -f $f, (Test-Path $f) }
"go           " + (Get-Command go     -ErrorAction Silent
"garble       " + (Get-Command garble -ErrorAction Silent
"cosign       " + (Get-Command cosign -ErrorAction Silent
"legacy go    " + (Test-Path C:\go1.20.14\bin\go.exe)
"legacy garble" + (Test-Path C:\go1.20.14\gobin\garble.ex
gpg --list-secret-keys releases@audspect.com
docker info --format 'Docker mem: {{.MemTotal}}'
```

All paths `True`, tools resolved, GPG key listed, Docker running → proceed.

---

5. Run the build

```
.\packaging\windows-build.ps1 `
  -Version    "1.8.5" `
  -Customer   "HDFC Bank" `
  -CustomerID "hdfc-prod-001" `
  -Days       311 `
```

```
  -WindowsCertThumbprint "<thumbprint-from-1h>"
```

~10–20 min; the orchestrator `garble -literals` stage is the slow/heavy one.

---

6. Verify outputs

```
dir dist\bas-install-1.8.5.zip, dist\bas-install-1.8.5.zi
dir hdfc-prod-001.lic
dir dist\pubkey.asc, dist\verify-sig.sh
```

The console summary prints the exact `scp` + client install commands. Ship the zip, its `.sha256`, `.asc`, `pubkey.asc`, `verify-sig.sh`, and the `.lic`.

---

7. Troubleshooting

Symptom	Cause
`ResourceExhausted … cannot allocate memory`, `Docker build failed`	Docker VM RAM too low for `garble -literals`
`Legacy agent build requires a dedicated go1.20.14 install at C:\go1.20.14 (not found)`	§1g not done

`Windows signing is required … but -WindowsCertThumbprint … is not set`	§1h missing
`cosign is not installed, and cosign signing is mandatory`	§1e missing
`cosign.key not found`	§2 not copied
`No usable GPG signing key for releases@audspect.com`	key not imported / keyboxd issue
`License private key not found …` (warning, no `.lic`)	`packaging\licensing\keys\private.pe` missing
`docker pull … failed` (caldera/headless-shell warning)	transient/network
`go`/`garble` not found	PATH
`BASAgent-Setup-*.exe is NN MB, expected ~14MB`	`go:embed` regression
stray `NativeCommandError` early in the run	leftover poisoned Go env from a prior failed run

> **License-only change** (renewal / new customer, no rebuild): don't rerun this whole script — use `bash packaging/licensing/issue-license.sh -customer "<name>" -id <id> -days <n>`, which signs against the same license key.

Notes

- The three §2 key files and the GPG/cert material are what make or break a customer build; the toolchain is easy, the secrets are the real work.
- Once the signing chain is set up on a machine, every later build there is just §5.